ASSIGNMENT 2

Advancing Front Hole Filling on 3D Meshes

Member:
Truong Tri Dung – MITIU25208
To Thanh Thuyen – MITIU25207
Nguyen Thanh Nghia – MITIU25205

1 Introduction

Hole filling is a fundamental problem in 3D mesh processing. Holes commonly arise during 3D scanning due to occlusion, sensor noise, or reflective surfaces. This assignment implements the **Advancing Front** method as a MeshLab C++ filter plugin to fill holes in triangle meshes.

The Advancing Front method was first introduced in the context of unstructured mesh generation [3] and later adapted for hole filling by Liepa [1] and Zhao et al. [2]. The core idea is to progressively grow new triangles from the hole boundary inward, selecting vertices by their local angle and applying geometric rules to produce well-shaped triangles that conform to the surrounding mesh structure.

2 Algorithm

The algorithm operates in three stages: hole detection, advancing front triangulation, and post-processing smoothing.

2.1 Hole Detection

A **boundary edge** is an edge that belongs to exactly one triangle. A **hole** is a closed loop of boundary edges.

1. Compute face–face adjacency for the entire mesh.
2. Mark each face edge as *border* if it has no adjacent face on that side.
3. Mark each vertex as *border* if it belongs to at least one border edge.
4. Starting from any unvisited border edge (v_i, v_j), trace the boundary loop: from v_j, find the next border edge leaving v_j that differs from the incoming edge. Continue until the loop closes back to v_i.

5. Record the ordered vertex indices, 3D positions, and per-vertex normals. Discard loops exceeding the `MaxHoleSize` threshold.

The tracing uses VCG's vertex–face iterator to enumerate all faces around the current vertex and locate the next border edge, ensuring correct traversal even for complex boundary configurations.

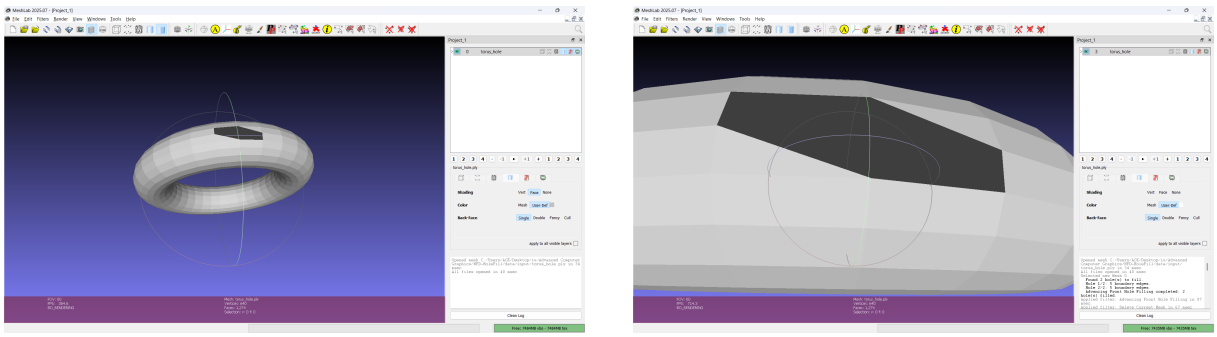
2.1.1 Visualizing Hole Boundaries

To assist with hole detection and visualization, the plugin includes a dedicated filter that highlights hole boundaries. This feature can be accessed via:

```
Filters > Selection > Detect Hole Boundary (Red)
```

This filter automatically identifies all boundary vertices that form holes in the mesh and colors them red (RGB: 255, 0, 0) for easy visualization. The implementation iterates through all vertices and marks those flagged as boundary vertices (`IsB()`) with red color. This visualization step is particularly useful for:

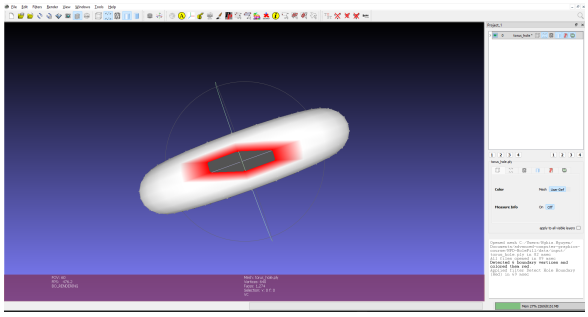
- Verifying that holes have been correctly detected
- Assessing the size and complexity of holes before filling
- Identifying multiple holes in a single mesh
- Confirming that the hole boundary forms a valid closed loop



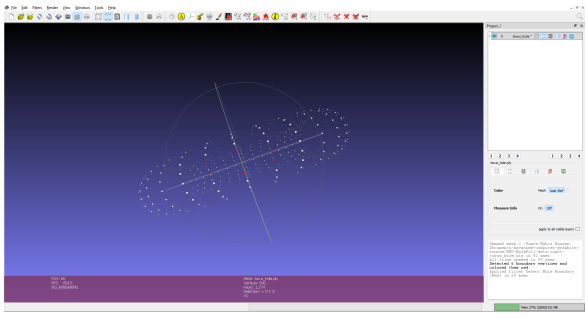
(a) Torus mesh with a hole

(b) Close-up of the hole boundary

Figure 1: Hole detection on a torus mesh. The boundary edges form a closed loop around the missing region.



(a) Overview of boundary detection



(b) Close-up of red boundary vertices

Figure 2: Boundary vertices highlighted in red using the `Detect Hole Boundary (Red)` filter. All boundary vertices are colored with RGB(255, 0, 0) for easy visualization before applying the hole filling algorithm.

2.2 Advancing Front Triangulation

The advancing front is initialized as the boundary loop. At each iteration, the front vertex with the smallest interior angle is selected and one of three rules is applied. The process continues until only three vertices remain, which form the final closing triangle.

2.2.1 Angle Computation

For each vertex v_i on the active front, the interior angle θ_i is computed between its two adjacent front edges:

$$\mathbf{e}_1 = v_{i-1} - v_i, \qquad \mathbf{e}_2 = v_{i+1} - v_i \tag{1}$$

$$\cos\theta_i = \frac{\mathbf{e}_1 \cdot \mathbf{e}_2}{\|\mathbf{e}_1\| \cdot \|\mathbf{e}_2\|} \tag{2}$$

$$\theta_i = \arccos(\cos\theta_i) \tag{3}$$

The vertex $v_{\min}$ with the smallest angle $\theta_{\min}$ is selected for processing. Selecting the smallest angle first ensures that narrow gaps are closed before wide ones, producing more uniform triangles and preventing self-intersection.

2.2.2 Rule 1: Small Angle ($\theta < 75$)

When the angle is small, the gap between the two neighboring edges is narrow. A single triangle closes it.

1. Create triangle $T = (v_{i-1},\ v_i,\ v_{i+1})$.
2. Remove v_i from the active front.
3. The front shrinks by one vertex.

No new vertices are introduced. This is the simplest and most common operation.

2.2.3 Rule 2: Medium Angle ($75 \leq \theta < 135$)

A medium angle cannot be closed cleanly with one triangle without producing a skewed element. A new vertex is inserted at the angle bisector.

1. Compute the bisector direction:

$$\mathbf{b} = \text{normalize}\left(\frac{\mathbf{e}_1}{\|\mathbf{e}_1\|} + \frac{\mathbf{e}_2}{\|\mathbf{e}_2\|}\right)$$

2. Place the new vertex:

$$v_{\text{new}} = v_i + \mathbf{b} \cdot L_{\text{avg}}$$

   where L_{avg} is the average edge length of the original boundary.

3. Create two triangles:

$$T_1 = (v_{i-1},\ v_i,\ v_{\text{new}}), \qquad T_2 = (v_i,\ v_{i+1},\ v_{\text{new}})$$

4. Replace v_i with v_{new} on the active front.

The bisector placement ensures the new vertex is equidistant from both edges, producing two roughly equal-sized triangles.

2.2.4 Rule 3: Large Angle ($\theta \geq 135$)

A large angle requires two new vertices to avoid elongated triangles. The angle span is divided into three sectors.

1. Compute the bisector $\mathbf{b}$ as in Rule 2.

2. Place two new vertices:

$$v_{\text{new}_1} = v_i + \text{normalize}\left(\frac{\mathbf{e}_1}{\|\mathbf{e}_1\|} + \mathbf{b}\right) \cdot L_{\text{avg}} \tag{4}$$

$$v_{\text{new}_2} = v_i + \text{normalize}\left(\frac{\mathbf{e}_2}{\|\mathbf{e}_2\|} + \mathbf{b}\right) \cdot L_{\text{avg}} \tag{5}$$

3. Create three triangles:

$$T_1 = (v_{i-1},\ v_i,\ v_{\text{new}_1}) \tag{6}$$
$$T_2 = (v_{\text{new}_1},\ v_i,\ v_{\text{new}_2}) \tag{7}$$
$$T_3 = (v_{\text{new}_2},\ v_i,\ v_{i+1}) \tag{8}$$

4. Replace v_i on the front with v_{new_1} and v_{new_2}.

Although the front temporarily grows by one vertex, subsequent iterations quickly close the newly created smaller angles via Rule 1.

2.2.5 Pseudocode

Algorithm 1 Advancing Front Hole Filling

Require: Boundary loop $B = [v_0, v_1, \ldots, v_{n-1}]$, average edge length L_{avg}
Ensure: Set of triangles $\mathcal{T}$ filling the hole

```
front ← B
while |front| > 3 do
    i ← arg min_k θ_k                                   ▷ Find smallest angle on front
    if θ_i < 75 then
        T ← T ∪ {(v_{i−1}, v_i, v_{i+1})}               ▷ Rule 1
        Remove v_i from front
    else if θ_i < 135 then
        v_new ← v_i + bisector(e_1, e_2) · L_avg        ▷ Rule 2
        T ← T ∪ {(v_{i−1}, v_i, v_new), (v_i, v_{i+1}, v_new)}
        Replace v_i with v_new in front
    else
        Compute v_new1, v_new2 at 1/3, 2/3 of angle span ▷ Rule 3
        T ← T ∪ {T_1, T_2, T_3}
        Replace v_i with [v_new1, v_new2] in front
    end if
end while
T ← T ∪ {(front[0], front[1], front[2])}                ▷ Close last triangle
Fix winding order if face normals oppose boundary normals
```

2.2.6 Termination and Safety

The loop terminates when exactly three vertices remain. A safety counter ($3\times$ initial boundary size) prevents infinite loops in degenerate cases. After triangulation, a winding-order check ensures all face normals align with the average boundary normal.

2.3 Laplacian Smoothing

The advancing front triangulation produces geometrically correct but potentially uneven surfaces. A uniform Laplacian smoothing pass refines interior vertex positions:

$$v_i' = \frac{1}{|N(i)|} \sum_{j \in N(i)} v_j \tag{9}$$

where $N(i)$ is the set of vertices adjacent to v_i. **Boundary vertices are held fixed** throughout the smoothing to preserve the exact boundary shape and ensure a seamless transition with the original mesh. The default is 3 iterations.

3 Results

The plugin was tested on meshes with artificially created holes of varying complexity.

3.1 Torus Mesh

Figure 3 shows the plugin's parameter dialog in MeshLab. Figures 4 and 5 show the torus mesh after applying the Advancing Front filter.

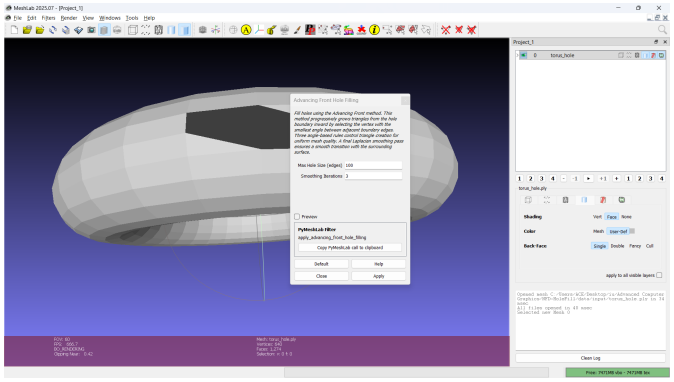
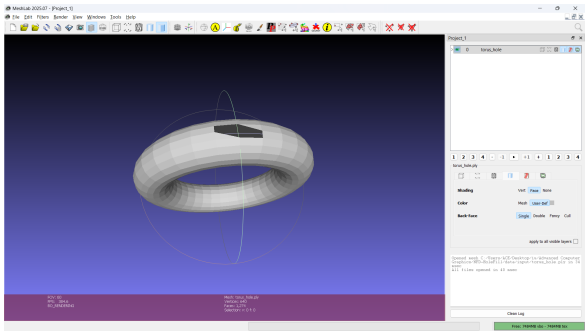
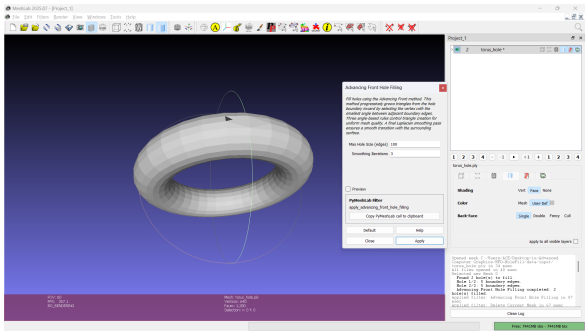


Figure 3: Plugin parameter dialog in MeshLab.

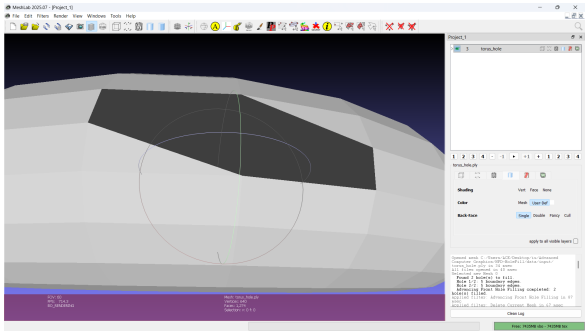


(a) Before: torus with hole

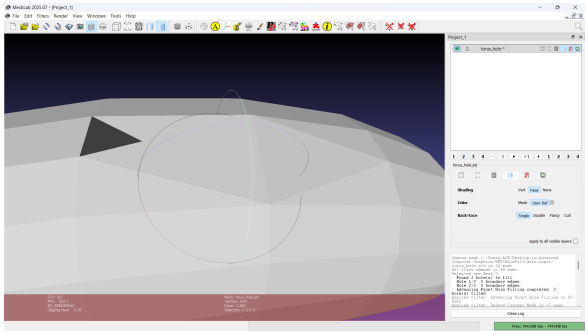


(b) After: hole filled

Figure 4: Torus hole filling result (overview).



(a) Before: close-up of hole

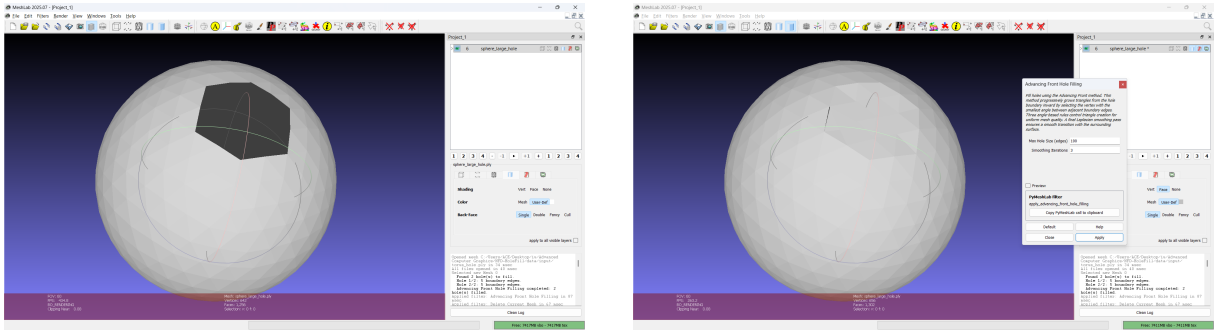


(b) After: close-up of filled region

Figure 5: Torus hole filling result (close-up). The generated triangles blend smoothly with the surrounding mesh after Laplacian smoothing.

3.2 Sphere Mesh (Large Hole)

The plugin was also tested on a sphere with a larger hole to evaluate performance on wider gaps where Rules 2 and 3 are more frequently applied.



(a) Before: sphere with large hole

(b) After: hole filled

Figure 6: Sphere large hole filling. The advancing front method produces uniform triangles even for larger holes by combining all three angle-based rules.

4 Implementation

4.1 Tools and Libraries

Component	Tool/Library	Purpose
C++ Framework	VCG Library	Mesh data structures, topology, normals
GUI/Plugin	Qt 5.15	MeshLab plugin interface
Build System	CMake + Ninja	Cross-platform build

4.2 Plugin Structure

- **filter_advancing_front.h** – Class declaration with `HoleBoundary` and `PatchMesh` data structures.
- **filter_advancing_front.cpp** – Full algorithm implementation (∼350 lines): `detectHoles()`, `advancingFrontFill()`, `smoothPatch()`, `mergePatchIntoMesh()`.
- **CMakeLists.txt** – Build configuration via `add_meshlab_plugin()`.

4.3 Parameters

Parameter	Type	Default	Description
MaxHoleSize	int	100	Max boundary edges per hole
SmoothingIterations	int	3	Laplacian smoothing passes

4.4 Key Implementation Details

Ocf Components. CMeshO uses VCG's Optional Component Framework (Ocf) for face–face and vertex–face adjacency. These must be explicitly enabled before topology operations via `m.face.EnableFFAdjacency()`, `m.face.EnableVFAdjacency()`, and `m.vert.EnableVFAdjacency()`. Omitting this causes a runtime exception.

Front Data Structure. The active front is a `std::vector<int>` of vertex indices. Boundary vertices occupy indices $0 \ldots n-1$; new interior vertices are appended. Rule 1 erases one element, Rule 2 replaces one, and Rule 3 replaces one and inserts one.

Average Edge Length. L_{avg} is computed once from the original boundary and used consistently for all new vertex placements, ensuring generated triangles match the surrounding mesh resolution.

5 How to Build and Run

1. Place plugin files in `meshlab-src/src/meshlabplugins/filter_advancing_front/`
2. Register the plugin in `meshlab-src/src/CMakeLists.txt`
3. Set up VS environment: `call vcvarsall.bat x64`
4. Build: `cmake -build build2 -target filter_advancing_front`
5. Run: `build2/src/distrib/meshlab.exe`
6. Import a mesh with holes, apply `Filters > Remeshing > Advancing Front Hole Filling`

See `BUILD_AND_RUN.md` for detailed instructions.

6 Conclusion

This assignment implements the Advancing Front hole filling algorithm as a MeshLab C++ plugin. The method uses three angle-based rules to progressively grow triangles from the boundary inward, producing well-shaped meshes with uniform triangle quality. The angle-based vertex selection ensures that narrow gaps are closed first, preventing self-intersection and maintaining good aspect ratios. Combined with Laplacian smoothing, the result is a smooth, seamless patch that blends naturally with the surrounding surface geometry. The results on both torus and sphere meshes demonstrate that the method handles holes of varying sizes and curvatures effectively.

References

[1] P. Liepa, "Filling holes in meshes," in *Proc. Eurographics Symposium on Geometry Processing*, pp. 200–205, 2003.

[2] W. Zhao, S. Gao, and H. Lin, "A robust hole-filling algorithm for triangular mesh," *The Visual Computer*, vol. 23, no. 12, pp. 987–997, 2007.

[3] S. H. Lo, "A new mesh generation scheme for arbitrary planar domains," *International Journal for Numerical Methods in Engineering*, vol. 21, no. 8, pp. 1403–1426, 1985.